PRACTICAL 1

AIM: Write a program to demonstrate the concept of Call by Value and Call by Reference.

Program:

```
#include<stdio.h>
#include<conio.h>
void Call(int a)
{
   a=a+10;
   printf("The Value of a inside the Call by Value Function:-%d\n",a);
}
void Ref(int *b)
{
   *b=*b+5;
   printf("The value of b inside the Call by Reference Function :- %d\n",*b);
}
void main()
 {
int x,y;
clrscr();
printf("Enter the value of x:-");
scanf("%d",&x);
printf("Enter the value of y:-");
scanf("%d",&y);
Call(x);
printf("After calling the function value of x outside the Call by Value function is :- %d\n",x);
Ref(&y);
 printf("After calling the function reference value of y outside the Call by Reference function is :- %d\n",y);
getch();
}
```

Output:

```
Enter the value of x:-5
Enter the value of y:-10
The Value of a inside the Call by Value Function:-15
After calling the function value of x outside the Call by Value function is :- 5
The value of b inside the Call by Reference Function :- 15
After calling the function reference value of y outside the Call by Reference function is :- 15
```

PRACTICAL 2

AIM: Write a program to perform addition, subtraction, and multiplication of matrices using arrays

Program:

```
#include <stdio.h>
#include<conio.h>
void main()
  {
    clrscr();
     int i,j,k,a[3][3],b[3][3],sum[3][3],sub[3][3],p[3][3];
     printf("Enter elements of Matrix A: ");
     printf("\n");
     for(i=0;i<3;i++)
     {
        for(j=0;j<3;j++)
        {
           scanf(" %d ",&a[i][j]);
        }
     }
     printf("Matrix A:");
     printf("\n");
     for(i=0;i<3;i++)
     {
        for(j=0;j<3;j++)
        {
           printf(" %d ",a[i][j]);
        }
        printf("\n");
     }
     printf("Enter elements of Matrix B: ");
```

```
printf("\n");
for(i=0;i<3;i++)
{
    for(j=0;j<3;j++)
    {
        scanf("%d",&b[i][j]);
    }
}
printf("Matrix B:");
printf("\n");
for(i=0;i<3;i++)
{
    for(j=0;j<3;j++)
    {
        printf(" %d ",b[i][j]);
    }
    printf("\n");
}
for(i=0;i<3;i++)
{
    for(j=0;j<3;j++)
    {
        sum[i][j]= a[i][j] + b[i][j];
    }
}
printf("Sum:");
printf("\n");
 for(i=0;i<3;i++)
{
    for(j=0;j<3;j++)
    {
        printf(" %d ",sum[i][j]);
```

```
    }
    printf("\n");
  }
  for(i=0;i<3;i++)
  {
     for(j=0;j<3;j++)
     {
        sub[i][j]= a[i][j] - b[i][j];
     }
  }
  printf("Sub:");
  printf("\n");
  for(i=0;i<3;i++)
  {
    for(j=0;j<3;j++)
    {
       printf(" %d ",sub[i][j]);
    }
    printf("\n");
  }

  for(i=0;i<3;i++)
  {
     for(j=0;j<3;j++)
     {
        for(k=0;k<3;k++)
        {
           p[i][j]=0;
           p[i][j]=p[i][j] + a[i][k]*b[k][j];
        }
     }
  }
```

```
        printf("Mul");
        printf("\n");
        for(i=0;i<3;i++)
        {
            for(j=0;j<3;j++)
            {
                printf(" %d ",p[i][j]);
            }
            printf("\n");
        }
getch();
    }
```

OUTPUT:

```
Enter elements of Matrix A:
1
2
3
4
5
6
7
8
9
10
Matrix A:
 1  2  3
 4  5  6
 7  8  9
Enter elements of Matrix B:
9
8
7
6
5
4
3
2
Matrix B:
 10  9  8
 7  6  5
 4  3  2
Sum:
 11  11  11
 11  11  11
 11  11  11
Sub:
 -9  -7  -5
 -3  -1  1
 3  5  7
Mul
 12  9  6
 24  18  12
 36  27  18
```

PRACTICAL: 3

AIM: Implement a stack using arrays and perform the following operations: Push, Pop, Peek, and Display

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5  // Maximum size of stack

int stack[MAX], top = -1;

// Function to Push element
void push() {
    int val;
    if (top == MAX - 1) {
        printf("\nStack Overflow! Cannot push element.\n");
    } else {
        printf("Enter value to push: ");
        scanf("%d", &val);
        top++;
        stack[top] = val;
        printf("%d pushed to stack.\n", val);
    }
}

// Function to Pop element
void pop() {
    if (top == -1) {
        printf("\nStack Underflow! No element to pop.\n");
    } else {
        printf("%d popped from stack.\n", stack[top]);
        top--;
    }
}

// Function to Peek element
```

```
void peek() {
    if (top == -1) {
        printf("\nStack is Empty.\n");
    } else {
        printf("Top element is: %d\n", stack[top]);
    }
}

// Function to Display elements
void display() {
    if (top == -1) {
        printf("\nStack is Empty.\n");
    } else {
        printf("\nStack elements are:\n");
        for (int i = top; i >= 0; i--) {
            printf("%d\n", stack[i]);
        }
    }
}

int main() {
    int choice;
    while (1) {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                peek();
                break;
```

```
            case 4:
                display();
                break;
            case 5:
                exit(0);
            default:
                printf("Invalid choice! Please try again.\n");
        }
    }
    return 0;
}
```

Output:

Output

```
--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 2
2 pushed to stack.

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 3
3 pushed to stack.
```

Output

```
3 pushed to stack.

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element is: 3

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4

Stack elements are:
3
2
```

PRACTICAL: 4

AIM: Write a program convent an infix expression to postfix using a stack using a C.

Program:

```
#include <stdio.h>
#include <ctype.h>
#define MAX 100

char stack[MAX];
int top=-1;

void push(char x)
{
   stack[++top]=x;
}

char pop()
{
   if(top==-1)
   return -1;
   else
   return stack[top--];

}

int precedence(char x)
{
   if(x=='(')
   return 0;
   if(x=='+'||x=='-')
```

```
    return 1;
    if(x=='*'||x=='/'||x=='%')
    return 2;
    if(x=='^')
    return 3;
    return 0;
}

int main()
{
    char exp[MAX];
    char*e,x;
    printf("Enter the Infix expression:");
    scanf("%s",&exp);
    e=exp;
    printf("Postfix Expression:");
    while(*e!='\0')
    {
        if(isalnum(*e))
        printf("%c",*e);
        else if(*e=='(')
        push(*e);
        else if(*e==')')
        {
            while((x=pop())!='(')
            printf("%c",x);
        }
        else
        {
            while(top!=-1&& precedence(stack[top])>=precedence(*e))
```

```
            printf("%c",pop());
            push(*e);
        }
        e++;
    }
    while(top!=-1)
    printf("%c",pop());
    return 0;
}
```

Output:

```
Enter the infix expression:AB-C+D
Postfix Expression: ABC-D+
```

PRACTICAL: 5

AIM: Write a program to evaluate a postfix expression using a stack.

Program:

```
#include <stdio.h>
#include <ctype.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int val)
{
    if (top == MAX - 1)
    {
        printf("Stack overflow!\n");
        return;
    }
    stack[++top] = val;
}

int pop()
{
    if (top == -1)
    {
        printf("Stack underflow!\n");
        return 0;
    }
```

```
    return stack[top--];
}

int evaluatePostfix(char* expr)
{
    int i, op1, op2;

    for (i = 0; expr[i] != '\0'; i++)
    {
        if (isdigit(expr[i]))
        {
            push(expr[i] - '0');  // convert char to int
        } else
        {
            op2 = pop();
            op1 = pop();
            switch (expr[i])
            {
                case '+': push(op1 + op2); break;
                case '-': push(op1 - op2); break;
                case '*': push(op1 * op2); break;
                case '/':
                    if (op2 == 0)
                    {
                        printf("Division by zero error!\n");
                        return 0;
                    }
                    push(op1 / op2);
                    break;
                default:
```

```
                printf("Unknown operator: %c\n", expr[i]);
                return 0;
            }
        }
    }

    return pop();
}

int main()
{
    char exp[MAX];

    printf("Enter a postfix expression (single-digit operands): ");
    scanf("%s", exp);  // Read expression as a string

    int result = evaluatePostfix(exp);
    printf("Result: %d\n", result);

    return 0;
}
```

Output:

```
Enter a postfix expression (single-digit operands): 1 + 3 - 5
Result: 1
```

PRACTICAL: 6

AIM: Implement the Tower of Hanoi problem using recursion.

Program;

```
#include <stdio.h>
void TowerOfHanoi(int n, char from, char to, char aux)
{
   if (n == 1)
   {
      printf("Move disk 1 from %c to %c\n", from, to);
      return;
   }
   TowerOfHanoi(n - 1, from, aux, to);
   printf("Move disk %d from %c to %c\n", n, from, to);
   TowerOfHanoi(n - 1, aux, to, from);
}

int main()
{
   int n = 3; // number of disks
   TowerOfHanoi(n, 'A', 'C', 'B');
   return 0;
}
```

Output:

```
Enter number of disks: 3
The sequence of moves to solve Tower of Hanoi with 3 disks:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```